

Q: ESP32 Weather Fetching System using OpenWeatherMap API and DHT22

Code

```
#include <WiFi.h>

#include <HTTPClient.h>

#include <ArduinoJson.h>

#include <DHT.h>

#include "config.h"

#define DHTPIN 33

#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {

  Serial.begin(115200);

  dht.begin();

  // Connect to WiFi

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  Serial.print("Connecting to WiFi");

  while (WiFi.status() != WL_CONNECTED) {

    delay(500);

    Serial.print(".");

  }

  Serial.println();

  Serial.println("WiFi Connected Successfully!");

  Serial.println();

  // OpenWeatherMap API URL

  String url =

    "http://api.openweathermap.org/data/2.5/weather?q=" +
```

```

String(CITY) +
"&appid=" +
String(API_KEY) +
"&units=metric";

HTTPClient http;

http.begin(url);

int httpResponseCode = http.GET();

if (httpResponseCode > 0) {

    String payload = http.getString();

    DynamicJsonDocument doc(2048);

    DeserializationError error = deserializeJson(doc, payload);

    if (error) {

        Serial.println("JSON Parsing Failed!");

        http.end();

        return;

    }

    // API Data

    String city = doc["name"];

    float apiTemp = doc["main"]["temp"];

    int apiHumidity = doc["main"]["humidity"];

    String weather = doc["weather"][0]["description"];

    // Local Sensor Data

    float localTemp = dht.readTemperature();

    float localHumidity = dht.readHumidity();

    if (isnan(localTemp) || isnan(localHumidity)) {

```

```
Serial.println("Failed to read DHT11 Sensor!");  
  
http.end();  
  
return;  
}
```

```
Serial.println("-----");  
Serial.println(" OpenWeatherMap Weather Report");  
Serial.println("-----");
```

```
Serial.print("City Name      : ");  
  
Serial.println(city);
```

```
Serial.print("API Temperature    : ");  
  
Serial.print(apiTemp);  
  
Serial.println(" °C");
```

```
Serial.print("Local Temperature   : ");  
  
Serial.print(localTemp);  
  
Serial.println(" °C");
```

```
Serial.print("Temperature Difference : ");  
  
Serial.print(localTemp - apiTemp);  
  
Serial.println(" °C");
```

```
Serial.print("API Humidity        : ");  
  
Serial.print(apiHumidity);
```

```
Serial.println(" %");
```

```
Serial.print("Local Humidity    : ");
```

```
Serial.print(localHumidity);
```

```
Serial.println(" %");
```

```
Serial.print("Humidity Difference  : ");
```

```
Serial.print(localHumidity - apiHumidity);
```

```
Serial.println(" %");
```

```
Serial.print("Weather Description  : ");
```

```
Serial.println(weather);
```

```
Serial.println("-----");
```

```
} else {
```

```
Serial.print("HTTP Request Failed. Error Code: ");
```

```
Serial.println(httpResponseCode);
```

```
}
```

```
http.end();
```

```
}
```

Q36. MQTT vs HTTP Comparison

Ans:

FEATURE	MQTT	HTTP
Architecture Model	Publish-Subscribe	Client-Server
Data Transfer Model	Broker-based	Request-Response
Power Consumption	Low	Higher
Latency	Low	Moderate
IOT Suitability	Excellent	Good
Security	TLS/SSL	HTTPS

For a 1000-node smart agriculture deployment, I would choose MQTT because it consumes less bandwidth, has lower power consumption, and is designed specifically for IoT communication. Its publish-subscribe architecture allows efficient communication between many sensor nodes and the cloud. MQTT also supports Quality of Service (QoS), ensuring reliable message delivery even when network conditions are unstable.

Q37. Wi-Fi Security for IoT

Ans : IoT devices are becoming increasingly common in homes, industries, and agriculture. Since these devices communicate over wireless networks, securing their Wi-Fi connection is essential to protect data and prevent unauthorized access.

One common attack on IoT devices is exploiting **WPA2 vulnerabilities**. If weak passwords or outdated firmware are used, attackers may gain access to the network. Another common attack is the **Evil Twin Attack**, where an attacker creates a fake Wi-Fi network with the same name as the legitimate one. The IoT device may unknowingly connect to the fake network, allowing the attacker to intercept sensitive data.

To improve security, ESP32 devices should use **WPA3** whenever it is available because it offers stronger encryption and better protection against password-based attacks. Another recommended practice is **certificate pinning**, which allows the ESP32 to verify that it is communicating with the intended server instead of a malicious one. This helps prevent man-in-the-middle attacks.

Developers should never hardcode Wi-Fi credentials directly in the source code. If the project is uploaded to GitHub or shared publicly, anyone can view the Wi-Fi username and

password. A better approach is to store these credentials in a separate configuration file, such as `config.h`, and add this file to `.gitignore` so it is not uploaded to the repository.

Regular firmware updates, strong passwords, secure communication using HTTPS, and proper authentication mechanisms are also important practices for building secure IoT systems.

References

1. Espressif Systems. *ESP32 Security Features*.
2. Wi-Fi Alliance. *WPA3 Security Overview*

Q38. LoRa & LPWAN – Long Range IoT

Ans:

LPWAN (Low Power Wide Area Network) is a wireless communication technology designed for IoT applications that require long-distance communication while consuming very little power. It is commonly used in smart agriculture, environmental monitoring, smart cities, and industrial automation.

One of the most popular LPWAN technologies is **LoRa (Long Range)**. LoRa uses a modulation technique called **Chirp Spread Spectrum (CSS)**, which allows signals to travel several kilometers while consuming minimal battery power. An important parameter in LoRa communication is the **Spreading Factor (SF)**. A higher spreading factor increases communication range and reliability but reduces the data transmission speed. This makes LoRa suitable for applications where small amounts of data are transmitted over long distances.

Comparison

Feature	LoRa	Zigbee	Z-wave
Range	2-15 km	10-100 m	30-100m
Power consumption	Very low	Low	Low
Data rate	Low	Medium	Low
Best Use	Low	Home Automation	Smart Homes

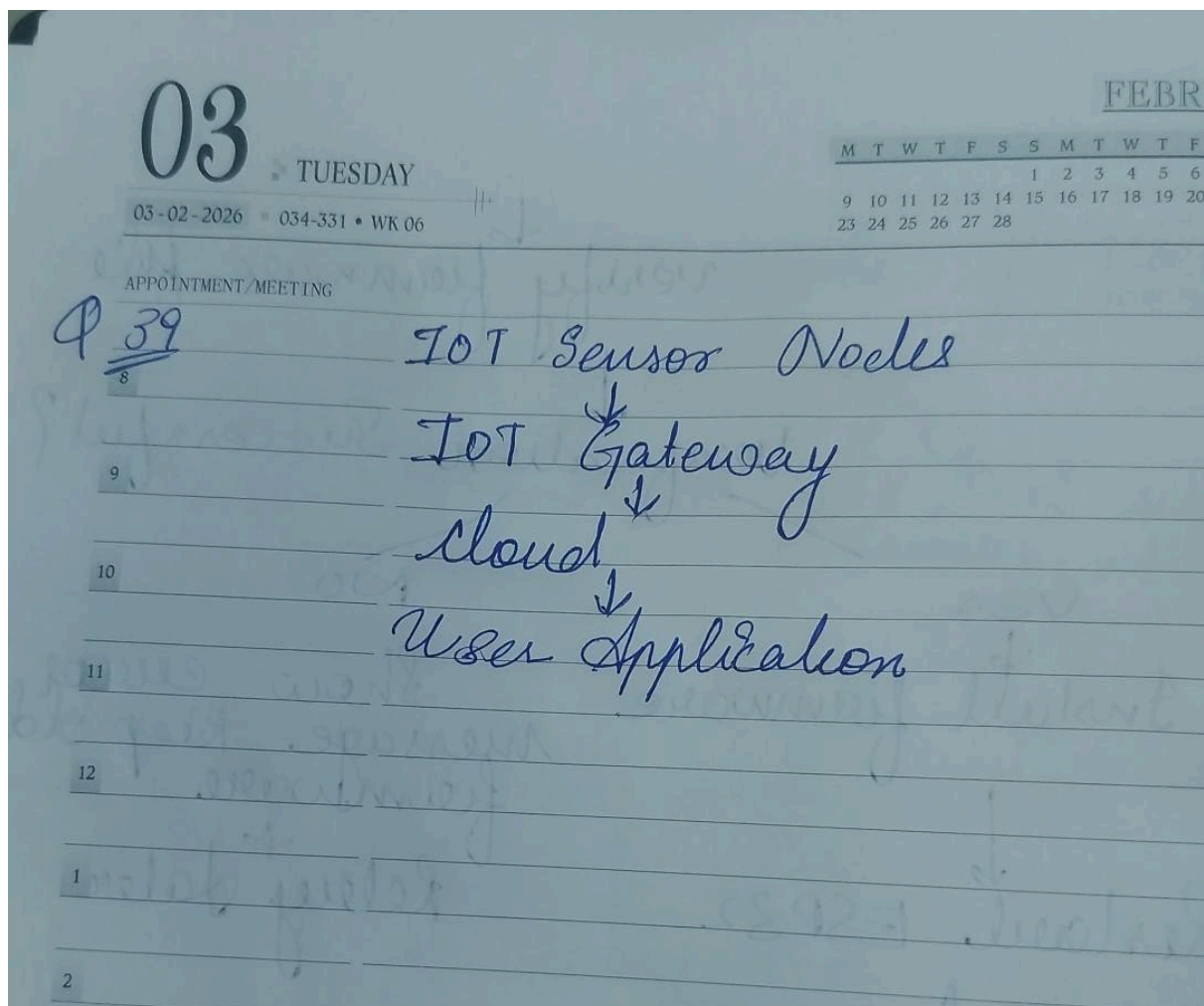
A real-world example in India is the use of LoRa-based smart water metering systems by municipal authorities. LoRa is also widely used in precision agriculture to monitor soil moisture, weather conditions, and irrigation systems over large farms.

Q39. What is an IoT Gateway?

Ans:

An **IoT Gateway** is a device that acts as a bridge between IoT sensor nodes and cloud platforms. It collects data from multiple sensors, processes or filters the information if required, converts different communication protocols, and securely forwards the data to cloud services. The gateway also allows cloud applications to send commands back to the connected devices.

DIAGRAM:



PROTOCOL USED:

LAYER	COMMON PROTOCOLS
Sensor -> gateway	MQTT,Zigbee,BLE,LoRa
Gateway->loud	MQTT,HTTPS
Cloud->User Application	HTTPS,WebSockets

Q40. Explain MQTT QoS Levels

Quality of Service (QoS) defines how reliably MQTT messages are delivered between devices and the broker.

QoS Level	Description
QoS 0(At most once)	The message is sent only once without any acknowledgment. It is the fastest but messages may be lost.
QoS 1 (At least once)	The sender retransmits the message until an acknowledgment is received. Messages are guaranteed to arrive but may be delivered more than once.
QoS 2(Exactly once)	The highest level of reliability. Messages are delivered exactly once using a four-step handshake, but this requires more bandwidth and processing time.

Which QoS level is suitable for a Water Level Monitoring System?

For an IoT-based water level monitoring system, **QoS 1** is the most suitable option. It ensures that important alerts, such as rising water levels or flood warnings, are delivered even if the network connection is temporarily unstable. Although duplicate messages may occasionally occur, receiving a duplicate alert is much safer than missing a critical warning.

QoS 1 provides a good balance between reliability, speed, and network efficiency, making it ideal for most monitoring applications.

Q48. Design Challenge: IoT for Jammu Smart City

Smart City Proposal for Jammu

Introduction

The Government of Jammu & Kashmir is working towards developing Jammu as a smarter and more sustainable city. Internet of Things (IoT) technology can help improve public services by enabling real-time monitoring, automation, and data-driven decision-making. As an IoT engineer, I propose implementing smart solutions for **Street Lighting**, **Air Quality Monitoring**, and **Waste Bin Monitoring**. These three applications address important urban challenges such as energy conservation, environmental pollution, and waste management. Together, they can improve the quality of life for citizens while reducing operational costs.

1) Smart Street Lighting

The Smart Street Lighting system will automatically control street lights based on ambient light levels and motion detection. An **LDR (Light Dependent Resistor)** will detect day and night conditions, while a **PIR sensor** will detect the movement of pedestrians or vehicles. During late-night hours, the street lights can remain dim to save energy and automatically increase brightness when movement is detected.

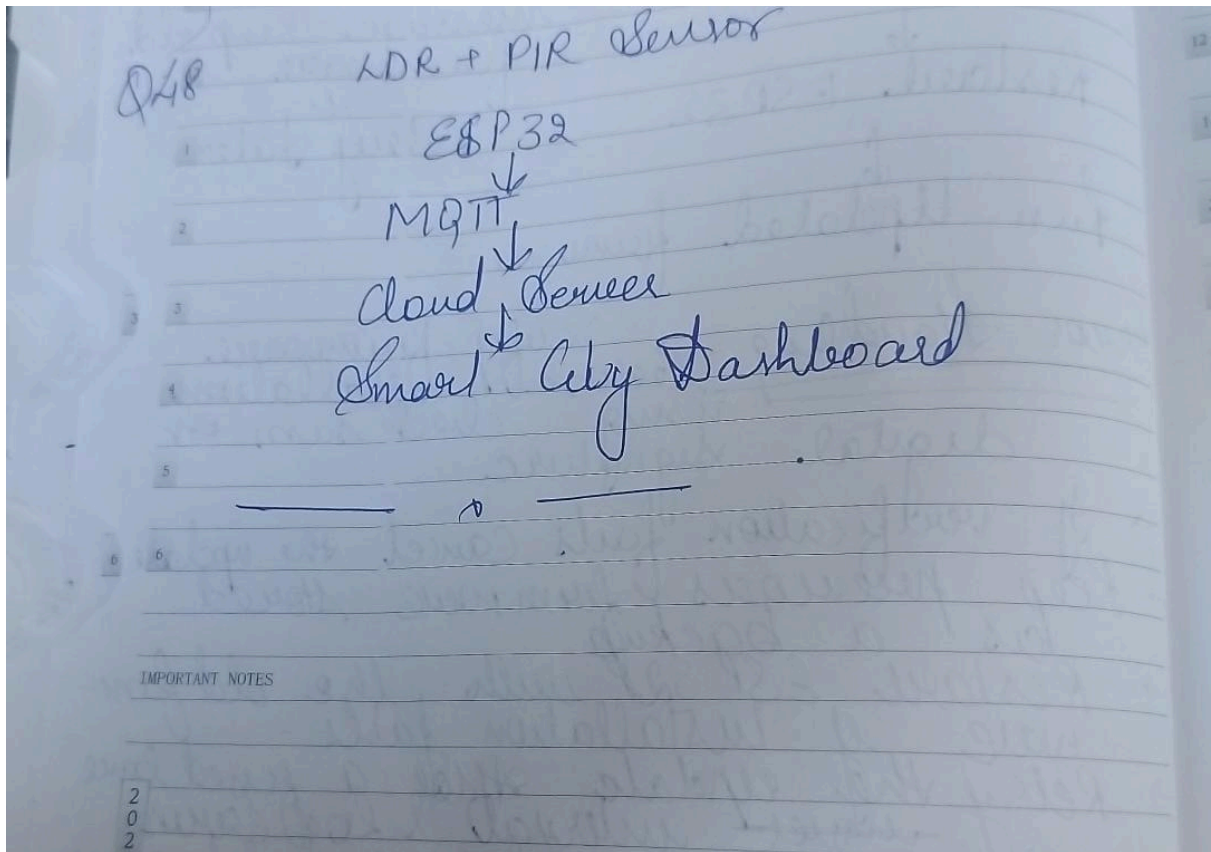
Sensors Required:

- LDR Sensor
- PIR Motion Sensor

Microcontroller & Connectivity:

- ESP32
- Wi-Fi (MQTT)

Data Flow:



Estimated Cost per Node:

components	Cost (INR)
esp32	500
LDR	30
PIR	120
Relay module	100
miscellenious	250
total	1000

This system will reduce electricity consumption and improve public safety by ensuring better lighting only when required.

Code:

```
#include <DHT.h>

#define DHTPIN 33

#define DHTTYPE DHT11

#define MQ 34

#define LED 25

DHT dht(DHTPIN,DHTTYPE);

void setup(){

  Serial.begin(115200);

  pinMode(LED,OUTPUT);

  dht.begin();

}

void loop(){

  int gas=analogRead(MQ);

  float temp=dht.readTemperature();

  float hum=dht.readHumidity();

  Serial.print("Gas : ");

  Serial.println(gas);

  Serial.print("Temp : ");

  Serial.println(temp);

  Serial.print("Humidity : ");

  Serial.println(hum);

  if(gas>600){

    digitalWrite(LED,HIGH);

    Serial.println("Poor Air Quality");

  }
```

```
else{  
  
digitalWrite(LED,LOW);  
  
Serial.println("Air Quality Good");  
  
}  
  
delay(2000);  
  
}
```

2) Smart Air Quality Monitoring

Air pollution is becoming a major concern in urban areas due to increasing traffic and industrial activities. An IoT-based Air Quality Monitoring System can continuously monitor harmful gases, temperature, and humidity at different locations across Jammu. The collected data can be displayed on a dashboard for municipal authorities to identify pollution hotspots and take timely action.

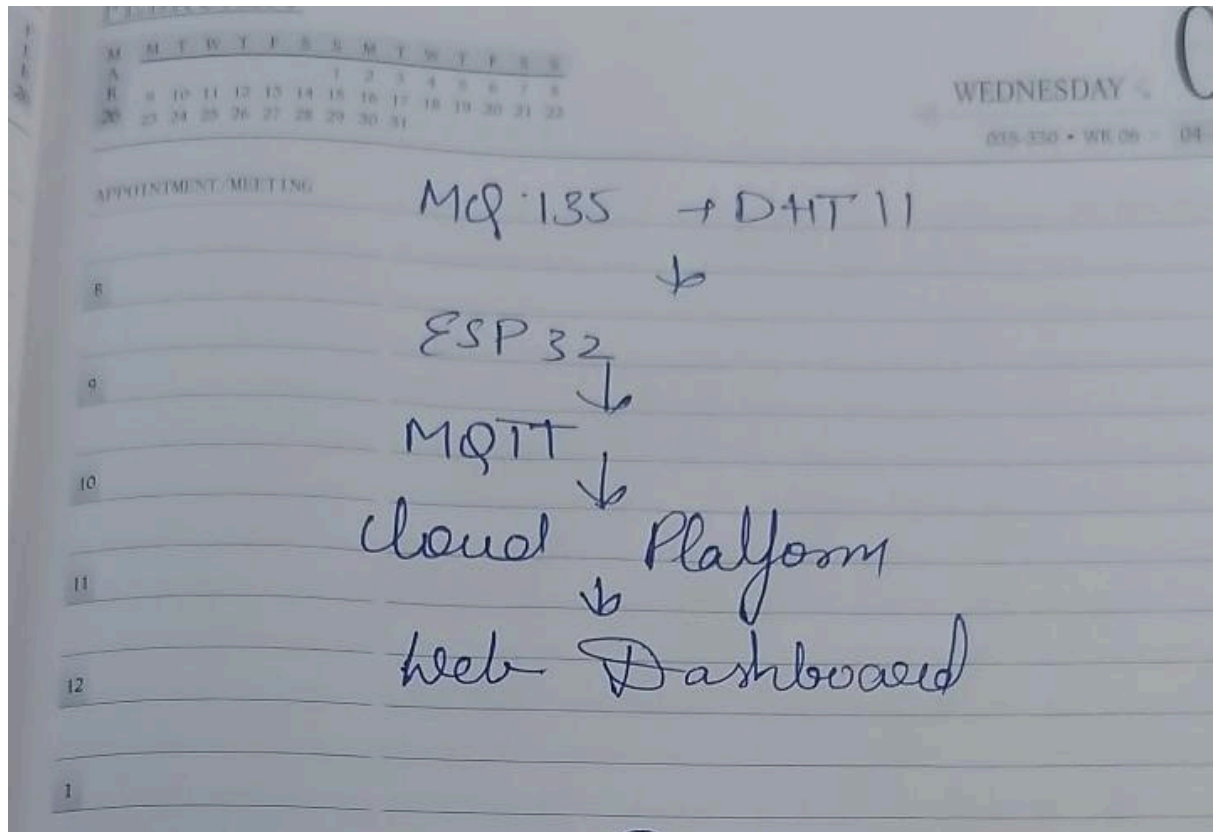
Sensors Required:

- MQ-135 Air Quality Sensor
- DHT11 Temperature & Humidity Sensor

Microcontroller & Connectivity:

- ESP32
- Wi-Fi (MQTT)

Data Flow:



component	cost(INR)
esp32	500
MQ 135	250
DHT 11	120
Power supply	250
Miscellaneous	180
total	1300

This system will provide real-time air quality information and help authorities issue pollution alerts whenever harmful gas levels exceed safe limits.

Code:

```
#define trigPin 5
```

```
#define echoPin 18
```

```
#define LED 25

long duration;

float distance;

void setup(){

  Serial.begin(115200);

  pinMode(trigPin,OUTPUT);

  pinMode(echoPin,INPUT);

  pinMode(LED,OUTPUT);

}

void loop()

  digitalWrite(trigPin,LOW);

  delayMicroseconds(2);

  digitalWrite(trigPin,HIGH);

  delayMicroseconds(10);

  digitalWrite(trigPin,LOW);

  duration=pulseIn(echoPin,HIGH);

  distance=duration*0.034/2;

  Serial.print("Distance : ");

  Serial.println(distance);

  if(distance<10){

    digitalWrite(LED,HIGH);

    Serial.println("Dustbin Full");

  }

  Else{

    digitalWrite(LED,LOW);

    Serial.println("Dustbin Available");
```

```
}  
  
delay(1000);  
  
}
```

3) Smart Waste Bin Monitoring

Overflowing garbage bins create hygiene problems and increase the risk of disease. A Smart Waste Bin Monitoring System can measure the fill level of waste bins using an ultrasonic sensor. When a bin reaches a predefined level, the system automatically sends an alert to the municipal waste collection department for timely collection.

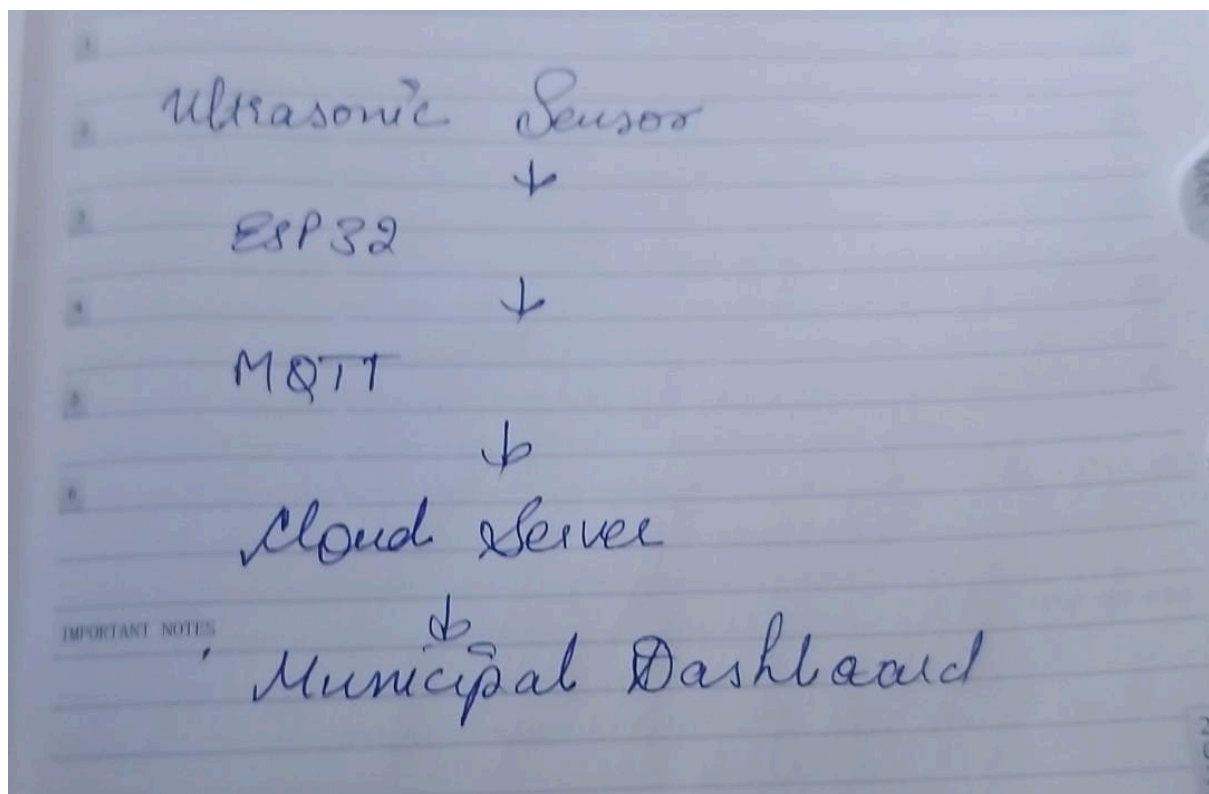
Sensors Required:

- HC-SR04 Ultrasonic Sensor

Microcontroller & Connectivity:

- ESP32
- Wi-Fi (MQTT)

Data Flow:



components	cost(INR)
Esp 32	500
HC-SR04	120
Power supply	250
Enclosure & Miscellaneous	330
total	1200

This system will optimize waste collection routes, reduce unnecessary vehicle movement, and help maintain cleaner public spaces.

Code:

```
#define trigPin 5

#define echoPin 18

#define LED 25

long duration;

float distance;

void setup(){

  Serial.begin(115200);

  pinMode(trigPin,OUTPUT);

  pinMode(echoPin,INPUT);

  pinMode(LED,OUTPUT);

}

void loop(){

  digitalWrite(trigPin,LOW);

  delayMicroseconds(2);

  digitalWrite(trigPin,HIGH);
```



```

delayMicroseconds(10);

digitalWrite(trigPin,LOW);

duration=pulseIn(echoPin,HIGH);

distance=duration*0.034/2;

Serial.print("Distance : ");

Serial.println(distance);

if(distance<10){

digitalWrite(LED,HIGH);

Serial.println("Dustbin Full")

}

else{

digitalWrite(LED,LOW);

Serial.println("Dustbin Available");

}

delay(1000);

}

```

Conclusion

The proposed IoT-based Smart City solutions for Jammu focus on improving energy efficiency, environmental monitoring, and waste management. By using ESP32-based sensor nodes connected through MQTT and cloud dashboards, city administrators can monitor conditions in real time and respond quickly to issues. These solutions are affordable, scalable, and suitable for deployment across different parts of the city. Implementing such smart systems will contribute to a cleaner, safer, and more sustainable Jammu while supporting the Government's vision of a modern Smart City.

application	sensor	controller	protocol	cost(INR)
Smart Street Lighting	LDR, PIR	ESP32	MQTT over Wi-Fi	1,000

Air Quality Monitoring	MQ-135, DHT11	ESP32	MQTT over Wi-Fi	1300
Smart Waste Bin Monitoring	HC-SR04	ESP32	MQTT over Wi-Fi	1200

Q49. Security Analysis of an IoT System

Security is one of the most important aspects of any IoT system because these devices communicate over the internet and often handle sensitive information. Poor security practices can allow attackers to steal data, control devices remotely, or disrupt system operations. The following are common security vulnerabilities found in IoT applications along with their risks and recommended solutions

Security Issue	Risk	Recommended Solution
Hardcoded Wi-Fi credentials	Anyone with access to the source code can obtain the Wi-Fi username and password.	Store credentials in a separate <code>config.h</code> file and exclude it from GitHub using <code>.gitignore</code> .
Unencrypted communication (HTTP)	Data can be intercepted or modified by attackers.	Use HTTPS or MQTT over TLS/SSL for encrypted communication.
No authentication	Unauthorized users can access or control the device.	Implement API keys, authentication tokens, or user login mechanisms.
Outdated firmware	Known vulnerabilities may be exploited by attackers.	Regularly update the firmware and apply security patches.
No input validation	Malicious or incorrect data can cause unexpected behavior.	Validate all incoming sensor values and user inputs before processing them.
Weak passwords	Devices can be compromised through brute-force attacks.	Use strong, unique passwords and enable WPA3 whenever possible.

Best Practices for Secure IoT Systems

- Store sensitive information such as Wi-Fi passwords and API keys in a separate configuration file.
- Use encrypted communication protocols such as HTTPS or MQTT with TLS.
- Keep the ESP32 firmware updated to the latest version.
- Use strong authentication methods before allowing remote access.
- Regularly monitor system logs to detect suspicious activity.
- Restrict physical access to IoT devices whenever possible.

Conclusion

A secure IoT system protects both the device and the user's data. By following good security practices such as encryption, authentication, secure credential storage, and regular software updates, IoT applications become more reliable and resistant to cyberattacks. Security should always be considered during the design and development of every IoT project.

Q50. Reflection Report

Reflection on IoT Summer School Learning

The IoT Summer School at IIT Jammu provided me with valuable hands-on experience in embedded systems, IoT programming, and real-world problem solving. Throughout the program, I learned how hardware and software work together to create smart and connected systems. The practical sessions helped me understand the complete workflow of designing, programming, testing, and documenting IoT projects.

During the internship, I became familiar with the **ESP32** and **Arduino IDE** for developing IoT applications. I learned how to interface different sensors such as the **DHT11** for temperature and humidity monitoring, **LDR** for light detection, **soil moisture sensors**, **ultrasonic sensors**, **PIR motion sensors**, and **LCD/OLED displays**. I also gained experience working with LEDs, buzzers, relays, and Bluetooth communication.

One of the most important skills I developed was understanding sensor data and using it to automate real-world tasks. Through various assignments and projects, the projects improved my understanding of sensor calibration, threshold-based decision making, and automation logic.

I also learned the importance of cloud connectivity in IoT. I explored how ESP32 devices can send data to cloud platforms and Google Sheets, and how communication protocols such as MQTT and HTTP are used for exchanging information between devices and servers. Additionally, I understood the role of REST APIs in accessing online data and the importance of secure communication using HTTPS and proper authentication.

Another valuable part of the internship was learning professional development practices. I created GitHub repositories to organize my weekly work, wrote README files for projects, maintained proper folder structures, and documented my work clearly. These practices helped me understand how collaborative software development is carried out in real engineering projects.

This Summer School has significantly increased my confidence in designing IoT solutions for practical applications such as smart homes, agriculture, healthcare, and smart cities. It has also strengthened my programming, debugging, and problem-solving skills. In the future, I would like to expand my knowledge by working on AI-enabled IoT systems, edge computing, and cloud-based analytics to build more intelligent and efficient solutions.

Overall, the IoT Summer School was an enriching learning experience that combined theory with practical implementation. It has provided me with a strong foundation in IoT development and motivated me to continue exploring this field through advanced projects and research.